VLM Prompt Engineering

Qwen3-VL이란?

- Alibaba Qwen 팀이 공개한 오픈소스 Vision-Language 모델
- Ollama로 로컬(온디바이스) 실행 가능 → API 비용 없이 실습
- 2B ~ 235B까지 다양한 크기, Instruct / Thinking 두 변종 제공

```
ollama pull qwen3-vl:4b
ollama run qwen3-vl:4b
```

> 이 강의의 예제 코드는 `qwen3-vl:4b` 기준으로 작성 (로컬 GPU 없어도 실습 가능한 크기)

VLM에게 "프롬프트 엔지니어링"이 왜 필요한가?

- VLM은 이미지를 "본" 뒤 **자연어로 응답을 생성**하는 모델
  - 같은 이미지라도 프롬프트에 따라 답의 형식·깊이·정확도가 크게 달라짐
- LLM 프롬프트 엔지니어링과 유사하지만, VLM만의 특성이 추가됨
  - 이미지 내 "어디를 볼지" 지시 가능 (영역, 객체, 텍스트 등)
  - 출력이 자유 서술형이 되기 쉬움 → **형식 고정**이 특히 중요
  - 모호한 질문 → 그럴듯하지만 틀린 답(할루시네이션) 위험

1. VLM Prompt Engineering

- Caption Prompt
- OCR Prompt
- Reasoning Prompt
- Structured Output

Caption Prompt (**이미지 캡셔닝**)

- 목적: 이미지 전체를 설명하는 자연어 문장 생성
- 프롬프트 설계 포인트
  - **범위 지정**: "한 문장으로" vs "상세하게 문단으로"
  - **관점 지정**: 객관적 사실만 vs 감성적 묘사
  - **용도 지정**: 대체 텍스트(alt text)용, SEO용, 상품 설명용 등

```
[나쁜 예] 이 이미지를 설명해줘.
[좋은 예] 이 상품 이미지를 온라인 쇼핑몰 상세페이지용
          alt 텍스트로 사용할 수 있게, 색상·소재·형태를 포함해
          한국어 한 문장으로 설명해줘.
```

OCR Prompt (텍스트 인식)

- 목적: 이미지 속 문자를 정확히 추출
- 프롬프트 설계 포인트
  - **"읽어줘"보다 "그대로 전사(transcribe)해줘"**가 정확도가 높음
  - 줄바꿈/표 구조 유지 여부를 명시
  - 언어를 명시 (다국어 혼재 시 오인식 방지)

```
[나쁜 예] 이 이미지에 뭐라고 써있어?
[좋은 예] 이미지 속 모든 텍스트를 줄바꿈까지 그대로 전사해줘.
          읽을 수 없는 글자는 [?]로 표시하고, 임의로 고치거나
          요약하지 마.
```

Reasoning Prompt (**시각적 추론**)

- 목적: 이미지 정보를 바탕으로 **계산/비교/판단**을 수행
- 프롬프트 설계 포인트
  - 단순 인식이 아니라 **추론 과정을 요구** (Chain-of-Thought)
  - "왜 그렇게 판단했는지 근거"를 함께 요청 → 할루시네이션 억제
  - 이미지에 없는 정보는 "모른다"고 답하도록 명시적으로 지시

```
이 차트에서 매출이 가장 높은 달과 가장 낮은 달의 차이를
계산해줘. 판단 근거(그래프의 어느 부분을 봤는지)를 먼저
설명한 뒤 답을 제시하고, 그래프에 없는 값은 추정하지 말고
"확인 불가"라고 답해.
```

Structured Output (구조화된 출력)

- 목적: 후속 처리(파싱/DB 저장/API 연동)가 가능한 **고정 포맷** 출력
- 프롬프트 설계 포인트
  - 출력 스키마(JSON 키/타입)를 **명시적으로 예시와 함께 제공**
  - "JSON만 출력, 설명 문장 금지"처럼 여백을 남기지 않기
  - 값이 없을 때 채울 기본값(`null` , `""` 등) 규칙을 지정

```
아래 JSON 스키마에 맞춰서만 답해. 다른 설명 문장은 출력하지 마.
{
  "title": "string",
  "colors": ["string"],
  "price": "number | null"
}
```

2. 이미지 정보 추출

- 상품 이미지 분석
- 객체 인식
- 속성 추출
- JSON 출력

상품 이미지 분석

- 이커머스에서 VLM 활용도가 가장 높은 영역 중 하나
- 입력: 상품 사진 1장
- 출력 예시: 카테고리, 색상, 소재, 특징 요약
- 프롬프트에 **쇼핑몰 카테고리 체계**를 함께 제시하면 분류 정확도 상승

```
이 상품 이미지를 분석해서 아래 카테고리 중 하나로 분류하고,
주요 색상과 특징을 요약해줘.
카테고리: [의류, 신발, 가방, 액세서리, 가전, 기타]
```

객체 인식 (Object Recognition)

- VLM은 별도 학습 없이도 이미지 내 **주요 객체를 텍스트로 나열** 가능
- 정밀한 좌표(bounding box)가 필요하면 Qwen3-VL의 grounding 기능 (좌표 출력) 활용 — 순수 캡션 모델보다 프롬프트가 더 구체적이어야 함

```
이미지에 보이는 모든 객체를 나열하고, 각 객체의 대략적인
위치(왼쪽 위/가운데/오른쪽 아래 등)를 함께 알려줘.
```

속성 추출 (Attribute Extraction)

- 객체를 "무엇인지"가 아니라 **"어떤 속성을 가지는지"** 뽑아내는 작업
- 상품 속성(색상, 사이즈, 패턴), 문서 속성(발행일, 금액) 등 도메인마다 다름
- 프롬프트에 **추출할 속성 목록을 미리 정의**해주는 것이 핵심

```
다음 속성만 추출해줘: 색상, 소재, 패턴, 넥라인, 소매기장.
해당 속성이 이미지에서 확인되지 않으면 "unknown"으로 표기해.
```

JSON 출력으로 파이프라인 연결

- 상품 분석 + 객체 인식 + 속성 추출을 **하나의 JSON 스키마**로 통합
- 이렇게 뽑은 JSON은 DB 적재, 검색 인덱싱, 추천 시스템 입력으로 바로 활용

```
{
  "category": "outer",
  "objects": ["coat", "button"],
  "attributes": {
    "color": "beige",
    "material": "wool",
    "pattern": "solid"
  }
}
```

3. OCR와 VLM 비교

- OCR 원리
- OCR 한계
- OCR + LLM
- OCR + VLM

OCR 원리

- **OCR(Optical Character Recognition)**: 이미지 속 문자 영역을 찾아 문자로 변환하는 전통적 파이프라인
- 일반적 단계
  i. 전처리 (이진화, 노이즈 제거, 기울기 보정)
  ii. 텍스트 영역 검출 (Text Detection)
  iii. 문자 인식 (Text Recognition, 예: CRNN)
  iv. 후처리 (맞춤법 보정 등)

OCR의 한계

- **레이아웃/의미를 이해하지 못함** — 글자만 뽑을 뿐, "이게 표인지 영수증인지"는 모름
- 손글씨, 왜곡된 텍스트, 저해상도에서 인식률 급락
- 표(Table) 구조, 항목 간 관계(예: 상품명-가격 매칭)는 별도 로직 필요
- 다국어/특수기호 혼재 시 오인식 다발

OCR + LLM

- OCR로 텍스트만 먼저 뽑고, 그 결과를 **LLM에 텍스트로 전달**해 해석
- 장점: 기존 OCR 자산을 재사용 가능
- 단점
  - OCR 단계 오류가 그대로 전파됨 (오탈자 → 잘못된 해석)
  - 레이아웃 정보(표의 행/열, 영수증의 항목 배치)가 텍스트로 평탄화되며 손실

```
[OCR 결과 텍스트] → LLM 프롬프트: "다음 텍스트에서 총 금액을 찾아줘"
```

OCR + VLM (**이미지를 직접 이해**)

- VLM은 **이미지를 그대로 입력**받아 텍스트+레이아웃을 함께 이해
- OCR 단계 없이 "이 영수증에서 총 금액이 얼마야?"를 바로 질의 가능

구분	OCR + LLM	VLM (**직접**)
입력	OCR 결과 텍스트	이미지 원본
레이아웃 정보	손실됨	유지됨
오류 전파	OCR 오류가 전파	단일 모델 오류만
표/양식 이해	별도 로직 필요	프롬프트만으로 가능
처리 속도	상대적으로 빠름	상대적으로 느림

> 실무에서는 상호 보완적으로 사용 — 정형 문서는 OCR+규칙, 비정형/복잡 레이아웃은 VLM

4. Document AI

- PDF 이해
- 계약서
- 영수증
- 표(Table) 분석

PDF 이해

- PDF를 **페이지 이미지로 렌더링**해 VLM에 직접 입력하는 방식이 핵심
  (텍스트 추출 라이브러리로 놓치는 레이아웃/그림/서명 등을 함께 파악)
- 활용 예: 목차 파악, 특정 섹션 요약, 표/그림 위치 질의

```
이 PDF 페이지의 핵심 내용을 3줄로 요약하고,
표나 그래프가 있다면 어떤 정보를 담고 있는지 설명해줘.
```

계약서 분석

- 요구되는 정확도가 매우 높은 도메인 → **근거 인용 필수**
- 프롬프트에 "원문 문구를 그대로 인용"하도록 지시해 검증 가능하게 만듦

```
이 계약서 이미지에서 계약 기간과 위약금 조항을 찾아,
관련 원문 문장을 그대로 인용한 뒤 요약해줘.
해당 조항이 없으면 "명시되어 있지 않음"이라고 답해.
```

영수증(Receipt) 분석

- 대표적인 **구조화 추출** 태스크: 상호명, 날짜, 품목, 단가, 총액
- 항목이 여러 줄인 표 형태이므로 VLM이 레이아웃을 이해해야 정확
- JSON 스키마 지정 + Few-shot 예시 조합이 특히 효과적

```
{
  "store": "string",
  "date": "YYYY-MM-DD",
  "items": [{"name": "string", "qty": "number", "price": "number"}],
  "total": "number"
}
```

표(Table) 분석

- 표는 "행-열 관계"가 의미의 핵심 → 단순 OCR로는 관계 파악이 어려움
- VLM 프롬프트 포인트
  - 표를 **마크다운 표 형식**으로 그대로 변환 요청 → 구조 보존 확인 용이
  - 특정 셀 값을 질의할 때는 "몇 번째 행/열"이 아니라 **행/열 헤더 이름**으로 지시

```
이 표를 마크다운 표 형식으로 그대로 옮겨 적어줘.
그 다음, "2024년" 열에서 가장 큰 값을 가진 행을 알려줘.
```

5. 차트 및 그래프 분석

- 막대그래프
- 선그래프
- 대시보드 분석

막대그래프(Bar Chart) 분석

- 확인 포인트: 축 레이블, 단위, 각 막대의 값, 최댓값/최솟값
- VLM은 막대 높이를 "픽셀 비교"로 추정하므로, **정확한 수치보다는 상대적 비교(더 크다/작다, 순위)에 강함**

```
이 막대그래프에서 값이 가장 큰 항목과 가장 작은 항목을 알려주고,
두 값의 차이를 추정해줘. 축 단위도 함께 확인해서 답변에 포함해.
```

선그래프(Line Chart) 분석

- 확인 포인트: 추세(상승/하락/변동), 변곡점, 구간별 기울기
- 프롬프트에 "추세를 설명"하도록 요청하면 요약 품질이 좋아짐

```
이 선그래프의 전체적인 추세를 설명하고, 값이 급격히
변한 구간이 있다면 x축 기준으로 어디인지 짚어줘.
```

대시보드 분석

- 여러 차트/KPI가 한 화면에 배치된 복합 이미지
- 프롬프트 설계 포인트
  - 한 번에 전체를 묻기보다 **차트별로 나눠 질의**하면 정확도 상승
  - "가장 주목해야 할 이상치/변화"를 먼저 찾도록 유도

```
이 대시보드에서 지난달 대비 가장 크게 변한 지표를 찾아
어느 차트에서 확인했는지와 함께 설명해줘.
```

6. UI 이해

- 웹페이지 분석
- Figma 화면 이해
- UX 분석

웹페이지 분석 (스크린샷 이해)

- 웹페이지 스크린샷을 입력받아 **레이아웃 구조**, **버튼/링크 위치**, **텍스트**를 파악
- 활용 예: 접근성 점검, 자동 QA, 스크린샷 기반 버그 리포트 요약

```
이 웹페이지 스크린샷에서 주요 내비게이션 메뉴 항목을 나열하고,
가장 눈에 띄는 CTA(Call To Action) 버튼이 어디에 있는지 설명해줘.
```

Figma 화면 이해

- 디자인 시안(목업)을 VLM으로 분석 → **디자인 스펙 문서화 자동화**에 활용
- 컴포넌트 명칭, 색상 코드, 간격 등은 정확도가 낮을 수 있어 **참고용**으로 활용

```
이 Figma 화면 캡처에서 화면을 구성하는 주요 컴포넌트
(헤더, 카드, 버튼 등)를 나열하고 전체적인 레이아웃 구조를 설명해줘.
```

UX 분석

- 화면 구조를 바탕으로 **사용성 관점의 피드백**을 요청하는 상위 태스크
- 프롬프트에 "어떤 사용자 기준으로 평가할지"를 명시하면 답변이 구체화됨

```
이 화면을 처음 방문한 사용자 입장에서 봤을 때,
다음 행동으로 무엇을 해야 할지 직관적으로 알 수 있는지 평가해줘.
개선이 필요하다면 구체적인 이유와 함께 제안해줘.
```